

## PROCESSI (Embryoio)

### 1

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash

### 2

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge will check for a hardcoded password over stdin : nrtkvcxr

### 3

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 1:utrwex-thvp

### 4

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : ocpvdc:eulskldvb

### 5

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge will check that input is redirected from a specific file path : /tmp/jekjwz - the challenge will check for a hardcoded password over stdin : wylocjms

### 6

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge will check that output is redirected to a specific file path : /tmp/xprfia

### 7

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge will check that the

environment is empty (except LC\_CTYPE, which is impossible to get rid of in some cases)

## 8

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript

## 9

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will check for a hardcoded password over stdin : kpmozkzx

## 10

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 1:zbzmaqopdr

## 11

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : bcarr:knixxoqfpi

## 12

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will check that input is redirected from a specific file path : /tmp/fuaakn - the challenge will check for a hardcoded password over stdin : koqnzypa

## 13

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will check that output is redirected to a specific file path : /tmp/umcqpri

## 14

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will check that the environment is empty (except LC\_CTYPE, which is impossible to get rid of in some cases)

## 15

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `ipython`

## 16

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `ipython` - the challenge will check for a hardcoded password over `stdin` : `momdoqba`

## 17

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `ipython` - the challenge will check that `argv[NUM]` holds value `VALUE` (listed to the right as `NUM:VALUE`) : `1:aeqsidferf`

## 18

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `ipython` - the challenge will check that `env[KEY]` holds value `VALUE` (listed to the right as `KEY:VALUE`) : `cjewri:vzkvteuinc`

## 19

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `ipython` - the challenge will check that input is redirected from a specific file path : `/tmp/lyyues` - the challenge will check for a hardcoded password over `stdin` : `tykxvxft`

## 20

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `ipython` - the challenge will check that output is redirected to a specific file path : `/tmp/rcleot`

## 21

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `ipython` - the challenge will check that the environment is empty (except `LC_CTYPE`, which is impossible to get rid of in some cases)

## 22

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python

## 23

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check for a hardcoded password over stdin : zzlmsgzc

## 24

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 1:kfcrhcrdqk

## 25

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : qjquoo:mraupvrrei

## 26

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that input is redirected from a specific file path : /tmp/krfelo - the challenge will check for a hardcoded password over stdin : wxzmmmdf

## 27

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that output is redirected to a specific file path : /tmp/khkdiq

## 28

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that the environment is empty (except LC\_CTYPE, which is impossible to get rid of in some cases)

## 29

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary

## 30

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check for a hardcoded password over stdin : gneipzst

## 31

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 1:pnsza-ovutr

## 32

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : bpuhsf:qkbswdgcat

## 33

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that input is redirected from a specific file path : /tmp/abvqxj - the challenge will check for a hardcoded password over stdin : bvavcsxv

## 34

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that output is redirected to a specific file path : /tmp/vugval

## 35

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that the environment is empty (except LC\_CTYPE, which is impossible to get rid of in some cases)

### **36**

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge checks for a specific process at the other end of stdout : cat

### **37**

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge checks for a specific process at the other end of stdout : grep

### **38**

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge checks for a specific process at the other end of stdout : sed

### **39**

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge checks for a specific process at the other end of stdout : rev

### **40**

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge checks for a specific process at the other end of stdin : cat - the challenge will check for a hardcoded password over stdin : uuhxwwpc

### **41**

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge checks for a specific process at the other end of stdin : rev - the challenge will check for a hardcoded password over stdin : wvsykkdl

### **42**

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge checks for a specific process at the other end of stdout : cat

### 43

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge checks for a specific process at the other end of stdout : grep

### 44

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge checks for a specific process at the other end of stdout : sed

### 45

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge checks for a specific process at the other end of stdout : rev

### 46

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge checks for a specific process at the other end of stdin : cat - the challenge will check for a hardcoded password over stdin : cqizkjlF

### 47

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge checks for a specific process at the other end of stdin : rev - the challenge will check for a hardcoded password over stdin : gfvxvtpa

### 48

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge checks for a specific process at the other end of stdout : cat

### 49

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge checks for a specific process at the other end of stdout : grep

## 50

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge checks for a specific process at the other end of stdout : sed

## 51

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge checks for a specific process at the other end of stdout : rev

## 52

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge checks for a specific process at the other end of stdin : cat - the challenge will check for a hardcoded password over stdin : wspivnce

## 53

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge checks for a specific process at the other end of stdin : rev - the challenge will check for a hardcoded password over stdin : nbdvfgzm

## 54

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdout : cat

## 55

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdout : grep

## 56

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdout : sed

## 57

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdout : rev

## 58

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdin : cat - the challenge will check for a hardcoded password over stdin : rvdvfsc

## 59

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdin : rev - the challenge will check for a hardcoded password over stdin : awbjdnel

## 60

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific process at the other end of stdout : cat

## 61

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific process at the other end of stdout : grep

## 62

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific process at the other end of stdout : sed

## 63

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific process at the other end of stdout : rev

## 64

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific process at the other end of stdin : cat - the challenge will check for a hardcoded password over stdin : rsczcige

## 65

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific process at the other end of stdin : rev - the challenge will check for a hardcoded password over stdin : dyqztbfx

## 66

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : find

## 67

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : find - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 1:ijixmywgg

## 68

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 124:vlplwgvgiv

## 70

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that the environment is empty (except LC\_CTYPE, which is impossible to get rid of in some cases) - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : 281:ubxgmbhfje

## 71

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that the environment is empty (except LC\_CTYPE, which is impossible to get rid of in some cases) - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 232:vwnpunjpdv - the challenge will

check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) :  
232:mlzuoxmqoz

## 72

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will check that input is redirected from a specific file path : fhfipd - the challenge will check that it is running in a specific current working directory : /tmp/oqjzag

## 73

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will check that it is running in a specific current working directory : /tmp/eewwhv - the challenge will check to make sure that the parent's parent CWD to be different than the challenge's CWD

## 74

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) :  
26:rohltuyquw

## 76

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that the environment is empty (except LC\_CTYPE, which is impossible to get rid of in some cases) - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : 219:gauthbuwjzg

## 77

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that the environment is empty (except LC\_CTYPE, which is impossible to get rid of in some cases) - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 330:nwzazucfbl - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : 319:fybniebwtz

## 78

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that input

is redirected from a specific file path : bdejfm - the challenge will check that it is running in a specific current working directory : /tmp/uyusyvv

## 79

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that it is running in a specific current working directory : /tmp/wfpsbh - the challenge will check to make sure that the parent's parent CWD to be different than the challenge's CWD

## 80

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 131:nukkm-zoorg

## 82

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that the environment is empty (except LC\_CTYPE, which is impossible to get rid of in some cases) - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : 147:jodrdrkgreh

## 83

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that the environment is empty (except LC\_CTYPE, which is impossible to get rid of in some cases) - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 167:smdznaebvx - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : 150:xhcfvpgqrn

## 84

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that input is redirected from a specific file path : pkexwn - the challenge will check that it is running in a specific current working directory : /tmp/fgguzo

## 85

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that it is

running in a specific current working directory : /tmp/ccxdwg - the challenge will check to make sure that the parent's parent CWD to be different than the challenge's CWD

## 86

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will force the parent process to solve a number of arithmetic problems : 1 - the challenge will use the following arithmetic operations in its arithmetic problems : +\* - the complexity (in terms of nested expressions) of the arithmetic problems : 1

## 87

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will force the parent process to solve a number of arithmetic problems : 5 - the challenge will use the following arithmetic operations in its arithmetic problems : +\*% - the complexity (in terms of nested expressions) of the arithmetic problems : 3

## 88

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 0:/tmp/ulijna

## 89

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 0:kobxyh

## 90

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will make sure that stdin is redirected from a fifo - the challenge will check for a hardcoded password over stdin : oxiambgi

## 91

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will make sure that stdout is a redirected from fifo

## 92

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will make sure that stdin is redirected from a fifo - the challenge will make sure that stdout is a redirected from fifo - the challenge will check for a hardcoded password over stdin : smhzmect

## 93

DA RIGUARDARE NON FUNZIONANTE

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will make sure that stdin is redirected from a fifo - the challenge will make sure that stdout is a redirected from fifo - the challenge will force the parent process to solve a number of arithmetic problems : 1 - the challenge will use the following arithmetic operations in its arithmetic problems : +\* - the complexity (in terms of nested expressions) of the arithmetic problems : 1

## 94

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will take input on a specific file descriptor : 261 - the challenge will check for a hardcoded password over stdin : oldqphed

## 95

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will take input on a specific file descriptor : 2 - the challenge will check for a hardcoded password over stdin : eawaijlo

## 96

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will take input on a specific file descriptor : 1 - the challenge will check for a hardcoded password over stdin : pgcgixlx

## 97

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will require the parent to send number of signals : 1

## 98

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will require the parent to send number of signals : 5

## 99

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will force the parent process to solve a number of arithmetic problems : 1 - the challenge will use the following arithmetic operations in its arithmetic problems : +\* - the complexity (in terms of nested expressions) of the arithmetic problems : 1

## 100

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will force the parent process to solve a number of arithmetic problems : 5 - the challenge will use the following arithmetic operations in its arithmetic problems : +\*% - the complexity (in terms of nested expressions) of the arithmetic problems : 3

## 101

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 0:/tmp/hnvcdj

## 102

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 0:tjnrrw

## 103

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will make sure that stdin is redirected from a fifo - the challenge will check for a hardcoded password over stdin : dueuwayn

## 104

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will make sure that stdout is a redirected from fifo

## 105

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will make sure that stdin is redirected from a fifo - the challenge will make sure that stdout is a redirected from fifo - the challenge will check for a hardcoded password over stdin : gcbxmdfp

## 106

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will make sure that stdin is redirected from a fifo - the challenge will make sure that stdout is a redirected from fifo - the challenge will force the parent process to solve a number of arithmetic problems : 1 - the challenge will use the following arithmetic operations in its arithmetic problems : +\* - the complexity (in terms of nested expressions) of the arithmetic problems : 1

## 107

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will take input on a specific file descriptor : 119 - the challenge will check for a hardcoded password over stdin : fransjsi

## 108

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will take input on a specific file descriptor : 2 - the challenge will check for a hardcoded password over stdin : pygjapje

## 109

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will take input on a specific file descriptor : 1 - the challenge will check for a hardcoded password over stdin : wbrlbfzz

## 110

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will require the parent to send number of signals : 1

### 111

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will require the parent to send number of signals : 5

### 112

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will force the parent process to solve a number of arithmetic problems : 1 - the challenge will use the following arithmetic operations in its arithmetic problems : +\* - the complexity (in terms of nested expressions) of the arithmetic problems : 1

### 113

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will force the parent process to solve a number of arithmetic problems : 5 - the challenge will use the following arithmetic operations in its arithmetic problems : +\*% - the complexity (in terms of nested expressions) of the arithmetic problems : 3

### 114

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 0:/tmp/rwzrwh

### 115

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 0:cvwbct

### 116

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will make sure that stdin is redirected from a fifo - the challenge will check for a hardcoded password over stdin : usixxqqw

### 117

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will make sure that stdout is a redirected from fifo

## 118

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will make sure that stdin is redirected from a fifo - the challenge will make sure that stdout is a redirected from fifo - the challenge will check for a hardcoded password over stdin : fpaugkjp

## 119

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will make sure that stdin is redirected from a fifo - the challenge will make sure that stdout is a redirected from fifo - the challenge will force the parent process to solve a number of arithmetic problems : 1 - the challenge will use the following arithmetic operations in its arithmetic problems : +\* - the complexity (in terms of nested expressions) of the arithmetic problems : 1

## 120

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will take input on a specific file descriptor : 341 - the challenge will check for a hardcoded password over stdin : oefxarzz

## 121

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will take input on a specific file descriptor : 2 - the challenge will check for a hardcoded password over stdin : aitlnbji

## 122

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will take input on a specific file descriptor : 1 - the challenge will check for a hardcoded password over stdin : thvueynh

## 123

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will take input on a specific file descriptor : 1 - the challenge will check for a hardcoded password over stdin : thvueynh

## 124

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will require the parent to send number of signals : 5

## 125

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will force the parent process to solve a number of arithmetic problems : 50 - the challenge will use the following arithmetic operations in its arithmetic problems : +\*&^%| - the complexity (in terms of nested expressions) of the arithmetic problems : 5

## 126

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will force the parent process to solve a number of arithmetic problems : 500 - the challenge will use the following arithmetic operations in its arithmetic problems : +\*&^%| - the complexity (in terms of nested expressions) of the arithmetic problems : 10

## 127

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will require the parent to send number of signals : 50

## 128

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will require the parent to send number of signals : 500

## 129

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge checks for a specific process at the other end of stdin : cat - the challenge checks for a specific process at the other end of stdout : cat - the challenge will force the parent process to solve a number of arithmetic problems : 50 - the challenge will use the following arithmetic operations in its arithmetic problems : +\*&^%| - the complexity (in terms of nested expressions) of the arithmetic problems : 5

### 130

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will force the parent process to solve a number of arithmetic problems : 50 - the challenge will use the following arithmetic operations in its arithmetic problems :  $+\&^{\%}$  - the complexity (in terms of nested expressions) of the arithmetic problems : 5

### 131

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will force the parent process to solve a number of arithmetic problems : 500 - the challenge will use the following arithmetic operations in its arithmetic problems :  $+\&^{\%}$  - the complexity (in terms of nested expressions) of the arithmetic problems : 10

### 132

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will require the parent to send number of signals : 50

### 133

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will require the parent to send number of signals : 500

### 134

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdin : cat - the challenge checks for a specific process at the other end of stdout : cat - the challenge will force the parent process to solve a number of arithmetic problems : 50 - the challenge will use the following arithmetic operations in its arithmetic problems :  $+\&^{\%}$  - the complexity (in terms of nested expressions) of the arithmetic problems : 5

### 135

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will force the parent process to solve a number of arithmetic problems : 50 - the challenge will use the following arithmetic operations in its arithmetic problems :  $+\&^{\%}$  - the complexity (in terms of nested expressions) of the arithmetic problems : 5

### 136

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will force the parent process to solve a number of arithmetic problems : 500 - the challenge will use the following arithmetic operations in its arithmetic problems : +\*^%| - the complexity (in terms of nested expressions) of the arithmetic problems : 10

### 137

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will require the parent to send number of signals : 50

### 138

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will require the parent to send number of signals : 500

### 139

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific process at the other end of stdin : cat - the challenge checks for a specific process at the other end of stdout : cat - the challenge will force the parent process to solve a number of arithmetic problems : 50 - the challenge will use the following arithmetic operations in its arithmetic problems : +\*^%| - the complexity (in terms of nested expressions) of the arithmetic problems : 5

### 140

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific (network) client process : shellscrip - the challenge will listen for input on a TCP port : 1916 - the challenge will force the parent process to solve a number of arithmetic problems : 5 - the challenge will use the following arithmetic operations in its arithmetic problems : +\*% - the complexity (in terms of nested expressions) of the arithmetic problems : 3

### 141

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific (network) client process : python - the challenge will listen for input on a TCP port : 1830 - the challenge will force the parent process to solve a number of arithmetic problems : 5 - the challenge will use the following arithmetic operations in its arithmetic problems : +\*% - the complexity (in terms of nested expressions) of the arithmetic problems : 3

## 142

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific (network) client process : binary - the challenge will listen for input on a TCP port : 1485 - the challenge will force the parent process to solve a number of arithmetic problems : 5 - the challenge will use the following arithmetic operations in its arithmetic problems : +\*% - the complexity (in terms of nested expressions) of the arithmetic problems : 3

## SETUID (Babysuid)

### 1

Welcome to /challenge/babysuid\_level1!

This challenge is part of a series of programs that exposes you to very simple programs that let you directly read the flag.

I just set the SUID bit on /usr/bin/cat. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level1) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/cat!

### 2

Welcome to /challenge/babysuid\_level2!

This challenge is part of a series of programs that exposes you to very simple programs that let you directly read the flag.

I just set the SUID bit on /usr/bin/more. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level2) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/more!

### 3

Welcome to /challenge/babysuid\_level3!

This challenge is part of a series of programs that exposes you to very simple programs that let you directly read the flag.

I just set the SUID bit on /usr/bin/less. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level3) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/less!

## 4

Welcome to /challenge/babysuid\_level4!

This challenge is part of a series of programs that exposes you to very simple programs that let you directly read the flag.

I just set the SUID bit on /usr/bin/tail. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level4) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/tail!

## 5

Welcome to /challenge/babysuid\_level5!

This challenge is part of a series of programs that exposes you to very simple programs that let you directly read the flag.

I just set the SUID bit on /usr/bin/head. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level5) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/head!

## 6

Welcome to /challenge/babysuid\_level6!

This challenge is part of a series of programs that exposes you to very simple programs that let you directly read the flag.

I just set the SUID bit on /usr/bin/sort. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level6) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/sort!

## 7

Welcome to /challenge/babysuid\_level7!

This challenge is part of a series of programs that shows you that an over-privileged editor is a very powerful tool, indeed.

I just set the SUID bit on /usr/bin/vim. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level7) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/vim!

## 8

Welcome to /challenge/babysuid\_level8!

This challenge is part of a series of programs that shows you that an over-privileged editor is a very powerful tool, indeed.

I just set the SUID bit on /usr/bin/emacs. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level8) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/emacs!

## 9

Welcome to /challenge/babysuid\_level9!

This challenge is part of a series of programs that shows you that an over-privileged editor is a very powerful tool, indeed.

I just set the SUID bit on /usr/bin/nano. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level9) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/nano!

## 10

Welcome to /challenge/babysuid\_level10!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/rev. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level10) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/rev!

## 11

Welcome to /challenge/babysuid\_level11!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/od. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level11) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/od!

## 12

Welcome to /challenge/babysuid\_level12!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/hd. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level12) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/hd!

## 13

Welcome to /challenge/babysuid\_level13!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/xxd. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level13) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/xxd!

## 14

Welcome to /challenge/babysuid\_level14!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/base32. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level14) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/base32!

## 15

Welcome to /challenge/babysuid\_level15!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/base64. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level15) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/base64!

## 16

Welcome to /challenge/babysuid\_level16!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/split. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level16) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/split!

## 17

Welcome to /challenge/babysuid\_level17!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on /usr/bin/gzip. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level17) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/gzip!

## 18

Welcome to /challenge/babysuid\_level18!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on /usr/bin/bzip2. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level18) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/bzip2

## 19

Welcome to /challenge/babysuid\_level19!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on /usr/bin/zip. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level19) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/zip!

## 20

Welcome to /challenge/babysuid\_level20!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on /usr/bin/tar. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level20) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/tar!

## 21

Welcome to /challenge/babysuid\_level21!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on /usr/bin/ar. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level21) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/ar!

## 22

Welcome to /challenge/babysuid\_level22!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on /usr/bin/cpio. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level22) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/cpio!

## 23

Welcome to /challenge/babysuid\_level23!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on /usr/bin/genisoimage. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level23) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/genisoimage!

## 24

Welcome to /challenge/babysuid\_level24!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/env. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level24) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/env!

## 25

Welcome to /challenge/babysuid\_level25!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/find. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level25) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/find!

## 26

Welcome to /challenge/babysuid\_level26!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/make. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level26) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/make!

## 27

Welcome to /challenge/babysuid\_level27!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/nice. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level27) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/nice!

## 28

Welcome to /challenge/babysuid\_level28!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/timeout. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level28) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/timeout!

## 29

Welcome to /challenge/babysuid\_level29!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/stdbuf. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level29) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/stdbuf!

## 30

Welcome to /challenge/babysuid\_level30!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/setarch. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level30) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/setarch!

## 31

Welcome to /challenge/babysuid\_level31!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/watch. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level31) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/watch!

## 32

Welcome to /challenge/babysuid\_level32!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/socat. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level32) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/socat!

## 33

Welcome to /challenge/babysuid\_level33!

This challenge is part of a series of programs that will require some light programming to read the flag..

I just set the SUID bit on /usr/bin/whiptail. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level33) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/whiptail!

## 34

Welcome to /challenge/babysuid\_level34!

This challenge is part of a series of programs that will require some light programming to read the flag..

I just set the SUID bit on /usr/bin/awk. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level34) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/awk!

## 35

Welcome to /challenge/babysuid\_level35!

This challenge is part of a series of programs that will require some light programming to read the flag..

I just set the SUID bit on /usr/bin/sed. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level35) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/sed!

## 36

Welcome to /challenge/babysuid\_level36!

This challenge is part of a series of programs that will require some light programming to read the flag..

I just set the SUID bit on /usr/bin/ed. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level36) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/ed!

## 37

Welcome to /challenge/babysuid\_level37!

This challenge is part of a series of programs that let you get the flag by doing tricks with permissions.

I just set the SUID bit on /usr/bin/chown. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level37) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/chown!

## 38

Welcome to /challenge/babysuid\_level38!

This challenge is part of a series of programs that let you get the flag by doing tricks with permissions.

I just set the SUID bit on /usr/bin/chmod. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level38) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/chmod!

## 39

Welcome to /challenge/babysuid\_level39!

This challenge is part of a series of programs that let you get the flag by doing tricks with permissions.

I just set the SUID bit on /usr/bin/cp. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level39) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/cp!

## 40

Welcome to /challenge/babysuid\_level40!

This challenge is part of a series of programs that let you get the flag by doing tricks with permissions.

I just set the SUID bit on /usr/bin/mv. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level40) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/mv!

## 41

Welcome to /challenge/babysuid\_level41!

This challenge is part of a series of programs that let you read the flag because they let you program anything.

I just set the SUID bit on /usr/bin/perl. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level41) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/perl!

## 42

Welcome to /challenge/babysuid\_level42!

This challenge is part of a series of programs that let you read the flag because they let you program anything.

I just set the SUID bit on /usr/bin/python. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level42) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/python!

## 43

Welcome to /challenge/babysuid\_level43!

This challenge is part of a series of programs that let you read the flag because they let you program anything.

I just set the SUID bit on /usr/bin/ruby. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level43) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/ruby!

## 44

Welcome to /challenge/babysuid\_level44!

This challenge is part of a series of programs that let you read the flag because they let you program anything.

I just set the SUID bit on /usr/bin/bash. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level44) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/bash!

## 45

Welcome to /challenge/babysuid\_level45!

This challenge is part of a series of programs that just straight up weren't designed to let you read files.

I just set the SUID bit on /usr/bin/date. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level45) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/date!

## 46

Welcome to /challenge/babysuid\_level46!

This challenge is part of a series of programs that just straight up weren't designed to let you read files.

I just set the SUID bit on /usr/bin/dmesg. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level46) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/dmesg!

## 47

Welcome to /challenge/babysuid\_level47!

This challenge is part of a series of programs that just straight up weren't designed to let you read files.

I just set the SUID bit on /usr/bin/wc. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level47) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/wc!

## 48

Welcome to /challenge/babysuid\_level48!

This challenge is part of a series of programs that just straight up weren't designed to let you read files.

I just set the SUID bit on /usr/bin/gcc. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level48) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/gcc!

## 49

Welcome to /challenge/babysuid\_level49!

This challenge is part of a series of programs that just straight up weren't designed to let you read files.

I just set the SUID bit on /usr/bin/as. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level49) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/as!

## 50

Welcome to /challenge/babysuid\_level50!

This challenge is part of a series of programs that just straight up weren't designed to let you read files.

I just set the SUID bit on /usr/bin/wget. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level50) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/wget!

## 51

Welcome to /challenge/babysuid\_level51!

This challenge is part of a series of programs that show you how dangerous it is to allow users to load their own code as plugins into the program (but figuring out how is the hard part!).

I just set the SUID bit on /usr/bin/ssh-keygen. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid\_level51) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/ssh-keygen!

## ASSEMBLY (Embryoasm)

### 1

Welcome to EmbryoASMLevel1

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers\_\_use.

In this level you will work with registers\_\_use! Please set the following: \* rdi = 0x1337

Please give me your assembly in bytes (up to 0x1000 bytes):

### 2

Welcome to EmbryoASMLevel2

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers\_\_use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers\_\_use. We will tell you which registers\_\_use are set beforehand and where you should put the result. In most cases, its rax.

Many instructions exist in x86 that allow you to do all the normal math operations on registers\_\_use and memory. For shorthand, when we say  $A += B$ , it really means,  $A = A + B$ . Here are some useful instructions:  $\text{add reg1, reg2} \Leftrightarrow \text{reg1} += \text{reg2}$   $\text{sub reg1, reg2} \Leftrightarrow \text{reg1} -= \text{reg2}$   $\text{imul reg1, reg2} \Leftrightarrow \text{reg1} *= \text{reg2}$   $\text{div}$  is a littler harder, we will discuss it later. Note: all 'regX' can be replaced by a constant or memory location

Do the following: \* add 0x331337 to rdi

We will now set the following in preparation for your code: rdi = 0xf6e

Please give me your assembly in bytes (up to 0x1000 bytes):

### 3

Welcome to EmbryoASMLevel3

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers\_\_use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers\_\_use. We will tell you which registers\_\_use are set beforehand and where you should put the result. In most cases, its rax.

Using your new knowledge, please compute the following:  $f(x) = mx + b$ , where:  $m = rdi$   $x = rsi$   $b = rdx$  Place the value into rax given the above. We will now set the following in preparation for your code:  $rdi = 0x19c4$   $rsi = 0x23dd$   $rdx = 0xf3a$

Please give me your assembly in bytes (up to 0x1000 bytes):

## 4

Welcome to EmbryoASMLLevel4

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers\_\_use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers\_\_use. We will tell you which registers\_\_use are set beforehand and where you should put the result. In most cases, its rax.

Recall division in x86 is more special than in normal math. Math in here is called integer math. This means everything, as it is now, is in the realm of whole looking numbers. As an example:  $10 / 3 = 3$  in integer math. Why? Because 3.33 gets rounded down to an integer. The relevant instructions for this level are: `mov rax, reg1; div reg2` Notice: to use this instruction you need to first load rax with the desired register you intended to be the divided. Then run `div reg2`, where reg2 is the divisor. This results in:  $rax = rdi / rsi$ ;  $rdx = \text{remainder}$  The quotient is placed in rax, the remainder is placed in rdx. Please compute the following:  $\text{speed} = \text{distance} / \text{time}$ , where:  $\text{distance} = rdi$   $\text{time} = rsi$  Place the value of speed into rax given the above. We will now set the following in preparation for your code:  $rdi = 0xebe$   $rsi = 0xb$

Please give me your assembly in bytes (up to 0x1000 bytes):

## 5

Welcome to EmbryoASMLLevel5

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers\_use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers `_use`. We will tell you which registers `_use` are set beforehand and where you should put the result. In most cases, its `rax`.

Modulo in assembly is another interesting concept! x86 allows you to get the remainder after doing a division on something. For instance:  $10 / 3 \rightarrow \text{remainder} = 1$  You can get the remainder of a division using the instructions introduced earlier through the `div` instruction. In most programming languages we refer to mod with the symbol '%'.

Please compute the following: rdi % rsi Place the value in rax.

We will now set the following in preparation for your code: `rdi = 0x3958dcfe` `rsi = 0x1f`

Please give me your assembly in bytes (up to 0x1000 bytes):

## 6

Welcome to EmbryoASMLLevel6

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with `registers_use`. We will tell you which `registers_use` are set beforehand and where you should put the result. In most cases, its `rax`.

Another cool concept in x86 is the independent access to lower register bytes. Each register in x86 is 64 bits in size, in the previous levels we have accessed the full register using `rax`, `rdi` or `rsi`. We can also access the lower bytes of each register using different register names. For example the lower 32 bits of `rax` can be accessed using `eax`, lower 16 bits using `ax`, lower 8 bits using `al`, etc. MSB

LSB +-----+ | rax | +-----+ +-----+ +-----+ -

| eax | +-----+-----+ | ax | +---+---+ | ah | al | +---+---+ Lower register bytes access is applicable to all registers\_\_use.

Using only the following instruction(s) mov Please compute the following: rax = rdi modulo 256 rbx = rsi modulo 65536

We will now set the following in preparation for your code: rdi = 0x37bd rsi = 0x6fc74346

Please give me your assembly in bytes (up to 0x1000 bytes):

## 7

Welcome to EmbryoASMLLevel7

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers\_\_use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers\_\_use. We will tell you which registers\_\_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with bit logic and operations. This will involve heavy use of directly interacting with bits stored in a register or memory location. You will also likely need to make use of the logic instructions in x86: and, or, not, xor.

Shifting in assembly is another interesting concept! x86 allows you to ‘shift’ bits around in a register. Take for instance, rax. For the sake of this example say rax only can store 8 bits (it normally stores 64). The value in rax is: rax = 10001010 If we shift the value once to the left: shl rax, 1 The new value is: rax = 00010100 As you can see, everything shifted to the left and the highest bit fell off and a new 0 was added to the right side. You can use this to do special things to the bits you care about. It also has the nice side affect of doing quick multiplication, division, and possibly modulo. Here are the important instructions: shl reg1, reg2 <=> Shift reg1 left by the amount in reg2 shr reg1, reg2 <=> Shift reg1 right by the amount in reg2 Note: all ‘regX’ can be replaced by a constant or memory location

Using only the following instructions: mov, shr, shl Please perform the following: Set rax to the 5th least significant byte of rdi i.e. rdi = | B7 | B6 | B5 | B4 | B3 | B2 | B1 | B0 | Set rax to the value of B4

We will now set the following in preparation for your code: rdi = 0x6a523ea9c0bb33f0

Please give me your assembly in bytes (up to 0x1000 bytes):

## 8

Welcome to EmbryoASMLLevel8

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers\_\_use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers\_\_use. We will tell you which registers\_\_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with bit logic and operations. This will involve heavy use of directly interacting with bits stored in a register or memory location. You will also likely need to make use of the logic instructions in x86: and, or, not, xor.

Bitwise logic in assembly is yet another interesting concept! x86 allows you to perform logic operation bit by bit on registers. For the sake of this example say registers only store 8 bits. The values in rax and rbx are: rax = 10101010 rbx = 00110011 'If we were to perform a bitwise AND of rax and rbx using the and rax, rbx instruction' the result would be calculated by ANDing each pair bits 1 by 1 hence why it's called a bitwise logic. So from left to right: 1 AND 0 = 0, 0 AND 0 = 0, 1 AND 1 = 1, 0 AND 1 = 0 ... Finally we combine the results together to get: rax = 00100010 Here are some truth tables for reference: AND OR XOR A | B | X A | B | X A | B | X —+—+— —+—+— —+—+— 0 | 0 | 0 0 | 0 | 0 0 | 0 | 0 0 | 1 | 0 0 | 1 | 1 1 | 0 | 0 1 | 0 | 1 1 | 1 | 1 1 | 1 | 1 1 | 1 | 1 1 | 1 | 0

Without using the following instructions: mov, xchg Please perform the following: rax = rdi AND rsi i.e. Set rax to the value of (rdi AND rsi)

We will now set the following in preparation for your code: rdi = 0x942825333ed839c3 rsi = 0x627219a74984b834

Please give me your assembly in bytes (up to 0x1000 bytes):

## 9

Welcome to EmbryoASMLLevel9

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers\_\_use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers\_\_use. We will tell you which registers\_\_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with bit logic and operations. This will involve heavy use of directly interacting with bits stored in a register or memory location. You will also likely need to make use of the logic instructions in x86: and, or, not, xor.

Using only the following instructions: and, or, xor Implement the following logic:

if x is even then y = 1 else y = 0 where: x = rdi y = rax

We will now set the following in preparation for your code: rdi = 0x358095a8

Please give me your assembly in bytes (up to 0x1000 bytes):

## 10

Welcome to EmbryoASMLLevel10

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers\_\_use. We will tell you which registers\_\_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with memory. This will require you to read or write to things stored linearly in memory. If you are confused, go look at the linear addressing module in 'ike. You may also be asked to dereference things, possibly multiple times, to things we dynamically put in memory for your use.

Up until now you have worked with registers as the only way for storing things, essentially variables like 'x' in math. Recall that memory can be addressed. Each address contains something at that location, like real addresses! As an example: the address '699 S Mill Ave, Tempe, AZ 85281' maps to the 'ASU Campus'. We would also say it points to 'ASU Campus'. We can represent this like: ['699 S Mill Ave, Tempe, AZ 85281'] = 'ASU Campus' The address is special because it is unique. But that also does not mean other address cant point to the same thing (as someone can have multiple houses). Memory is exactly the same! For instance, the address in memory that your code is stored (when we take it from you) is 0x400000. In x86 we can access the thing at a memory location, called dereferencing, like so: mov rax, [some\_\_address] <=> Moves the thing at 'some\_\_address' into rax This also works with things in registers: mov rax, [rdi]

`<=>` Moves the thing stored at the address of what `rdi` holds to `rax` This works the same for writing: `mov [rax], rdi` `<=>` Moves `rdi` to the address of what `rax` holds. So if `rax` was `0xdeadbeef`, then `rdi` would get stored at the address `0xdeadbeef`: `[0xdeadbeef] = rdi` Note: memory is linear, and in x86, it goes from `0 - 0xffffffff` (yes, huge).

Please perform the following: 1. Place the value stored at `0x404000` into `rax` 2. Increment the value stored at the address `0x404000` by `0x1337` Make sure the value in `rax` is the original value stored at `0x404000` and make sure that `[0x404000]` now has the incremented value.

We will now set the following in preparation for your code: `[0x404000] = 0x12371a`

Please give me your assembly in bytes (up to `0x1000` bytes):

## 11

Welcome to EmbryoASMLLevel11

To interact with any level you will send raw bytes over `stdin` to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with `registers_use`. We will tell you which `registers_use` are set beforehand and where you should put the result. In most cases, its `rax`.

In this level you will be working with memory. This will require you to read or write to things stored linearly in memory. If you are confused, go look at the linear addressing module in 'ike. You may also be asked to dereference things, possibly multiple times, to things we dynamically put in memory for your use.

Recall that registers in `x86_64` are 64 bits wide, meaning they can store 64 bits in them. Similarly, each memory location is 64 bits wide. We refer to something that is 64 bits (8 bytes) as a quad word. Here is the breakdown of the names of memory sizes: \* Quad Word = 8 Bytes = 64 bits \* Double Word = 4 bytes = 32 bits \* Word = 2 bytes = 16 bits \* Byte = 1 byte = 8 bits In `x86_64`, you can access each of these sizes when dereferencing an address, just like using bigger or smaller register accesses: `mov al, [address]` `<=>` moves the least significant byte from address to `rax` `mov ax, [address]` `<=>` moves the least significant word from address to `rax` `mov eax, [address]` `<=>` moves the least significant double word from address to `rax` `mov rax, [address]` `<=>` moves the full quad word from address to `rax` Remember that moving only into `al` for instance does not fully clear the upper bytes.

Please perform the following: 1) Set `rax` to the byte at `0x404000` 2) Set `rbx` to the word at `0x404000` 3) Set `rcx` to the double word at `0x404000` 4) Set `rdx` to the quad word at `0x404000`

We will now set the following in preparation for your code: `[0x404000] = 0x1dc397`  
Please give me your assembly in bytes (up to 0x1000 bytes):

## 12

Welcome to EmbryoASMLLevel12

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with `registers_use`. We will tell you which `registers_use` are set beforehand and where you should put the result. In most cases, its `rax`.

In this level you will be working with memory. This will require you to read or write to things stored linearly in memory. If you are confused, go look at the linear addressing module in 'ike. You may also be asked to dereference things, possibly multiple times, to things we dynamically put in memory for your use.

It is worth noting, as you may have noticed, that values are stored in reverse order of how we represent them. As an example, say: `[0x1330] = 0x00000000dead0de` If you examined how it actually looked in memory, you would see: `[0x1330] = 0xde 0xc0 0xad 0xde 0x00 0x00 0x00 0x00` This format of storing things in 'reverse' is intentional in x86, and its called Little Endian.

For this challenge we will give you two addresses created dynamically each run. The first address will be placed in `rdi`. The second will be placed in `rsi`. Using the earlier mentioned info, perform the following: 1. set `[rdi] = 0xdeadbeef00001337` 2. set `[rsi] = 0xc0ffee0000` Hint: it may require some tricks to assign a big constant to a dereferenced register. Try setting a register to the constant then assigning that register to the derefed register.

We will now set the following in preparation for your code: `[0x4044d8] = 0xffffffffffffff` `[0x404a40] = 0xffffffffffffff` `rdi = 0x4044d8` `rsi = 0x404a40`

Please give me your assembly in bytes (up to 0x1000 bytes):

## 13

Welcome to EmbryoASMLLevel13

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of

formulaic operation with `registers__use`. We will tell you which `registers__use` are set beforehand and where you should put the result. In most cases, its `rax`.

In this level you will be working with memory. This will require you to read or write to things stored linearly in memory. If you are confused, go look at the linear addressing module in 'ike. You may also be asked to dereference things, possibly multiple times, to things we dynamically put in memory for your use.

Recall that memory is stored linearly. What does that mean? Say we access the quad word at 0x1337: `[0x1337] = 0x00000000deadbeef` The real way memory is layed out is byte by byte, little endian: `[0x1337] = 0xef` `[0x1337 + 1] = 0xbe` `[0x1337 + 2] = 0xad` ... `[0x1337 + 7] = 0x00` What does this do for us? Well, it means that we can access things next to each other using offsets, like what was shown above. Say you want the 5th *byte* from an address, you can access it like: `mov al, [address+4]` Remember, offsets start at 0.

Perform the following: 1. load two consecutive quad words from the address stored in `rdi` 2. calculate the sum of the previous steps quad words. 3. store the sum at the address in `rsi`

We will now set the following in preparation for your code: `[0x404198] = 0xc8bbb` `[0x4041a0] = 0x5f897` `rdi = 0x404198` `rsi = 0x404670`

Please give me your assembly in bytes (up to 0x1000 bytes):

## 14

Welcome to EmbryoASMLLevel14

To interact with any level you will send raw bytes over `stdin` to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with `registers__use`. We will tell you which `registers__use` are set beforehand and where you should put the result. In most cases, its `rax`.

In this level you will be working with the Stack, the memory region that dynamically expands and shrinks. You will be required to read and write to the Stack, which may require you to use the `pop` & `push` instructions. You may also need to utilize `rsp` to know where the stack is pointing.

In these levels we are going to introduce the stack. The stack is a region of memory, that can store values for later. To store a value a on the stack we use the `push` instruction, and to retrieve a value we use `pop`. The stack is a last in first out (LIFO) memory structure this means the last value pushed in the first value popped. Imagine unloading plates from the dishwasher let's say there are 1 red, 1 green, and 1 blue. First we place the red one in the cabinet, then the green on top of the red, then the blue. Out stack of plates would look like: Top

—> Blue Green Bottom -> Red Now if we wanted a plate to make a sandwich we would retrieve the top plate from the stack which would be the blue one that was last into the cabinet, ergo the first one out.

Replace the top value of the stack with (top value of the stack - rdi).

We will now set the following in preparation for your code: rdi = 0xb57 (stack) [0x7ffff1ffff8] = 0x29e58f7c

Please give me your assembly in bytes (up to 0x1000 bytes):

## 15

Welcome to EmbryoASMLLevel15

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers\_use. We will tell you which registers\_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with the Stack, the memory region that dynamically expands and shrinks. You will be required to read and write to the Stack, which may require you to use the pop & push instructions. You may also need to utilize rsp to know where the stack is pointing.

In this level we are going to explore the last in first out (LIFO) property of the stack.

Using only following instructions: push, pop Swap values in rdi and rsi. i.e. If to start rdi = 2 and rsi = 5 Then to end rdi = 5 and rsi = 2

We will now set the following in preparation for your code: rdi = 0x3860f368 rsi = 0x16bd319c

Please give me your assembly in bytes (up to 0x1000 bytes):

## 16

Welcome to EmbryoASMLLevel16

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers\_use. We will tell you which registers\_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with the Stack, the memory region that dynamically expands and shrinks. You will be required to read and write to the Stack, which may require you to use the pop & push instructions. You may also need to utilize `rsp` to know where the stack is pointing.

In the previous levels you used push and pop to store and load data from the stack however you can also access the stack directly using the stack pointer. The stack pointer is stored in the special register `rsp`. `rsp` always stores the memory address to the top of the stack, i.e. the memory address of the last value pushed. Similar to the memory levels we can use `[rsp]` to access the value at the memory address in `rsp`.

Without using pop please calculate the average of 4 consecutive quad words stored on the stack. Store the average on the top of the stack. Hint: `RSP+0x??` Quad Word A `RSP+0x??` Quad Word B `RSP+0x??` Quad Word C `RSP` Quad Word D `RSP-0x??` Average

We will now set the following in preparation for your code: (stack) `[0x7ffff200000:0x7ffff1fffe0]` = `['0x36811c7a', '0x229edb15', '0x7b46a74', '0x2d7def8']` (list of things)

Please give me your assembly in bytes (up to 0x1000 bytes):

## 17

Welcome to EmbryoASMLLevel17

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers\_use. We will tell you which registers\_use are set beforehand and where you should put the result. In most cases, its `rax`.

In this level you will be working with control flow manipulation. This involves using instructions to both indirectly and directly control the special register `rip`, the instruction pointer. You will use instructions like: `jmp`, `call`, `cmp`, and the like to implement requests behavior.

Earlier, you learned how to manipulate data in a pseudo-control way, but x86 gives us actual instructions to manipulate control flow directly. There are two major ways to manipulate control flow: 1. through a jump; 2. through a call. In this level, you will work with jumps. There are two types of jumps: 1. Unconditional jumps 2. Conditional jumps Unconditional jumps always trigger and are not based on the results of earlier instructions. As you know, memory locations can store data and instructions. Your code will be stored at `0x4000ad` (this will change each run). For all jumps, there are three types: 1. Relative jumps 2. Absolute jumps 3. Indirect jumps In this level we will ask you to do

both a relative jump and an absolute jump. You will do a relative jump first, then an absolute one. You will need to fill space in your code with something to make this relative jump possible. We suggest using the `nop` instruction. It's 1 byte and very predictable. Useful instructions for this level is: `jmp (reg1 | addr | offset) ; nop` Hint: for the relative jump, lookup how to use `labels` in x86.

Using the above knowledge, perform the following: Create a two jump trampoline:  
1. Make the first instruction in your code a `jmp`  
2. Make that `jmp` a relative jump to 0x51 bytes from its current position  
3. At 0x51 write the following code:  
4. Place the top value on the stack into register `rdi`  
5. `jmp` to the absolute address 0x403000

We will now set the following in preparation for your code: - Loading your given code at: 0x4000ad - (stack) [0x7ffff1ffff8] = 0x74

Please give me your assembly in bytes (up to 0x1000 bytes):

## 18

Welcome to EmbryoASMLLevel18

To interact with any level you will send raw bytes over `stdin` to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with control flow manipulation. This involves using instructions to both indirectly and directly control the special register `rip`, the instruction pointer. You will use instructions like: `jmp`, `call`, `cmp`, and the like to implement requests behavior.

We will be testing your code multiple times in this level with dynamic values! This means we will be running your code in a variety of random ways to verify that the logic is robust enough to survive normal use. You can consider this as normal dynamic value se

We will now introduce you to conditional jumps—one of the most valuable instructions in x86. In higher level programming languages, an if-else structure exists to do things like: if x is even: `is_even = 1` else: `is_even = 0` This should look familiar, since its implementable in only bit-logic. In these structures, we can control the programs control flow based on dynamic values provided to the program. Implementing the above logic with `jumps` can be done like so:

```
; assume rdi = x, rax is output
; rdx = rdi mod 2
mov rax, rdi
mov rsi, 2
div rsi
; remainder is 0 if even
cmp rdx, 0
; jump to not_even code is its not 0
```

```

jne not_even
; fall through to even code
mov rbx, 1
jmp done
; jump to this only when not_even
not_even:
mov rbx, 0
done:
mov rax, rbx
; more instructions here

```

Often though, you want more than just a single ‘if-else’. Sometimes you want two if checks, followed by an else. To do this, you need to make sure that you have control flow that ‘falls-through’ to the next `if` after it fails. All must jump to the same `done` after execution to avoid the else. There are many jump types in x86, it will help to learn how they can be used. Nearly all of them rely on something called the ZF, the Zero Flag. The ZF is set to 1 when a `cmp` is equal. 0 otherwise.

Using the above knowledge, implement the following: if `[x]` is 0x7f454c46:  $y = [x+4] + [x+8] + [x+12]$  else if `[x]` is 0x00005A4D:  $y = [x+4] - [x+8] - [x+12]$  else:  $y = [x+4] * [x+8] * [x+12]$  where: `x = rdi`, `y = rax`. Assume each dereferenced value is a signed dword. This means the values can start as a negative value at each memory position. A valid solution will use the following at least once: `jmp` (any variant), `cmp`

We will now run multiple tests on your code, here is an example run: - (data) [0x404000] = {4 random dwords} - `rdi = 0x404000`

Please give me your assembly in bytes (up to 0x1000 bytes):

## 19

Welcome to EmbryoASMLLevel19

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with control flow manipulation. This involves using instructions to both indirectly and directly control the special register `rip`, the instruction pointer. You will use instructions like: `jmp`, `call`, `cmp`, and the like to implement requests behavior.

We will be testing your code multiple times in this level with dynamic values! This means we will be running your code in a variety of random ways to verify that the logic is robust enough to survive normal use. You can consider this as normal dynamic value se

The last set of jump types is the indirect jump, which is often used for switch statements in the real world. Switch statements are a special case of if-statements that use only numbers to determine where the control flow will go. Here is an example: `switch(number): 0: jmp do_thing_0 1: jmp do_thing_1 2: jmp do_thing_2 default: jmp do_default_thing` The switch in this example is working on **number**, which can either be 0, 1, or 2. In the case that **number** is not one of those numbers, default triggers. You can consider this a reduced else-if type structure. In x86, you are already used to using numbers, so it should be no surprise that you can make if statements based on something being an exact number. In addition, if you know the range of the numbers, a switch statement works very well. Take for instance the existence of a jump table. A jump table is a contiguous section of memory that holds addresses of places to jump. In the above example, the jump table could look like: `[0x1337] = address of do_thing_0 [0x1337+0x8] = address of do_thing_1 [0x1337+0x10] = address of do_thing_2 [0x1337+0x18] = address of do_default_thing` Using the jump table, we can greatly reduce the amount of cmps we use. Now all we need to check is if **number** is greater than 2. If it is, always do: `jmp [0x1337+0x18]` Otherwise: `jmp [jump_table_address + number * 8]` Using the above knowledge, implement the following logic: if rdi is 0: `jmp 0x403035` else if rdi is 1: `jmp 0x4030de` else if rdi is 2: `jmp 0x4031ec` else if rdi is 3: `jmp 0x40329b` else: `jmp 0x403360` Please do the above with the following constraints: - assume rdi will NOT be negative - use no more than 1 cmp instruction - use no more than 3 jumps (of any variant) - we will provide you with the number to 'switch' on in rdi. - we will provide you with a jump table base address in rsi.

Here is an example table: `[0x404135] = 0x403035` (addrs will change) `[0x40413d] = 0x4030de` `[0x404145] = 0x4031ec` `[0x40414d] = 0x40329b` `[0x404155] = 0x403360`

Please give me your assembly in bytes (up to 0x1000 bytes):

## 20

Welcome to EmbryoASMLLevel20

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers\_use. We will tell you which registers\_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with control flow manipulation. This involves using instructions to both indirectly and directly control the special register **rip**, the instruction pointer. You will use instructions like: `jmp`, `call`, `cmp`, and the like to implement requests behavior.

In a previous level you computed the average of 4 integer quad words, which was

a fixed amount of things to compute, but how do you work with sizes you get when the program is running? In most programming languages a structure exists called the for-loop, which allows you to do a set of instructions for a bounded amount of times. The bounded amount can be either known before or during the programs run, during meaning the value is given to you dynamically. As an example, a for-loop can be used to compute the sum of the numbers 1 to n: `sum = 0 i = 1 for i <= n: sum += i i += 1`

Please compute the average of n consecutive quad words, where: rdi = memory address of the 1st quad word rsi = n (amount to loop for) rax = average computed

We will now set the following in preparation for your code: `- [0x4040d8:0x404290] = {n qwords}` - rdi = 0x4040d8 - rsi = 55

Please give me your assembly in bytes (up to 0x1000 bytes):

## 21

Welcome to EmbryoASMLLevel21

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with control flow manipulation. This involves using instructions to both indirectly and directly control the special register `rip`, the instruction pointer. You will use instructions like: `jmp`, `call`, `cmp`, and the like to implement requests behavior.

We will be testing your code multiple times in this level with dynamic values! This means we will be running your code in a variety of random ways to verify that the logic is robust enough to survive normal use. You can consider this as normal dynamic value se

In previous levels you discovered the for-loop to iterate for a *number* of times, both dynamically and statically known, but what happens when you want to iterate until you meet a condition? A second loop structure exists called the while-loop to fill this demand. In the while-loop you iterate until a condition is met. As an example, say we had a location in memory with adjacent numbers and we wanted to get the average of all the numbers until we find one bigger or equal to 0xff: `average = 0 i = 0 while x[i] < 0xff: average += x[i] i += 1 average /= i`

Using the above knowledge, please perform the following: Count the consecutive non-zero bytes in a contiguous region of memory, where: rdi = memory address of the 1st byte rax = number of consecutive non-zero bytes Additionally, if rdi = 0, then set rax = 0 (we will check)! An example test-case, let: rdi = 0x1000 [0x1000] = 0x41 [0x1001] = 0x42 [0x1002] = 0x43 [0x1003] = 0x00 then: rax = 3 should be set

We will now run multiple tests on your code, here is an example run: - (data) [0x404000] = {10 random bytes}, - rdi = 0x404000

Please give me your assembly in bytes (up to 0x1000 bytes):

## 22

Welcome to EmbryoASMLLevel22

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will be testing your code multiple times in this level with dynamic values! This means we will be running your code in a variety of random ways to verify that the logic is robust enough to survive normal use. You can consider this as normal dynamic value se

In this level you will be working with functions! This will involve manipulating both ip control as well as doing harder tasks than normal. You may be asked to utilize the stack to save things and call other functions that we provide you.

In previous levels you implemented a while loop to count the number of consecutive non-zero bytes in a contiguous region of memory. In this level you will be provided with a contiguous region of memory again and will loop over each performing a conditional operation till a zero byte is reached. All of which will be contained in a function!

A function is a callable segment of code that does not destroy control flow. Functions use the instructions “call” and “ret”.

The “call” instruction pushes the memory address of the next instruction onto the stack and then jumps to the value stored in the first argument.

Let’s use the following instructions as an example: 0x1021 mov rax, 0x400000 0x1028 call rax 0x102a mov [rsi], rax

1. call pushes 0x102a, the address of the next instruction, onto the stack.
2. call jumps to 0x400000, the value stored in rax. The “ret” instruction is the opposite of “call”. ret pops the top value off of the stack and jumps to it. Let’s use the following instructions and stack as an example:

| Stack ADDR | VALUE |
|------------|-------|
|------------|-------|

|        |                                                                                           |
|--------|-------------------------------------------------------------------------------------------|
| 0x103f | mov rax, rdx RSP + 0x8 0xdeadbeef 0x1042 ret RSP + 0x0 0x0000102a ret will jump to 0x102a |
|--------|-------------------------------------------------------------------------------------------|

Please implement the following logic: str\_lower(src\_addr): rax = 0 if src\_addr != 0: while [src\_addr] != 0x00: if [src\_addr] <= 0x5a: [src\_addr] = foo([src\_addr]) rax += 1 src\_addr += 1 foo is provided at 0x403000. foo takes a single argument as a value

An important note is that `src_addr` is an address in memory (where the string is located) and `[src_addr]` refers to the byte that exists at `src_addr`.

Therefore, the function `foo` expects a byte as its first argument, and returns a byte.

We will now run multiple tests on your code, here is an example run: - (data) `[0x404000] = {10 random bytes}`, - `rdi = 0x404000`

Please give me your assembly in bytes (up to 0x1000 bytes):

## 23

Welcome to EmbryoASMLLevel23

To interact with any level you will send raw bytes over `stdin` to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will be testing your code multiple times in this level with dynamic values! This means we will be running your code in a variety of random ways to verify that the logic is robust enough to survive normal use. You can consider this as normal dynamic value se

In this level you will be working with functions! This will involve manipulating both ip control as well as doing harder tasks than normal. You may be asked to utilize the stack to save things and call other functions that we provide you.

In the previous level, you learned how to make your first function and how to call other functions. Now we will work with functions that have a function stack frame. A function stack frame is a set of pointers and values pushed onto the stack to save things for later use and allocate space on the stack for function variables. First, let's talk about the special register `rbp`, the Stack Base Pointer. The `rbp` register is used to tell where our stack frame first started. As an example, say we want to construct some list (a contiguous space of memory) that is only used in our function. The list is 5 elements long, each element is a dword. A list of 5 elements would already take 5 registers, so instead, we can make pace on the stack! The assembly would look like:

```
~~~~~ ; setup the base  
of the stack as the current top mov rbp, rsp ; move the stack 0x14 bytes (5 * 4)  
down ; acts as an allocation sub rsp, 0x14 ; assign list2 = 1337 mov eax, 1337 mov  
[rbp-0x8], eax ; do more operations on the list ... ; restore the allocated space  
mov rsp, rbp ret ~~~~~
```

Notice how `rbp` is always used to restore the stack to where it originally was. If we don't restore the stack after use, we will eventually run out TM. In addition, notice how we subtracted from `rsp` since the stack grows down. To make it have more space, we subtract the space we need. The `ret` and `call` still works the same. It is assumed that you will never pass a stack address across functions, since, as you can see from the above use, the stack can be overwritten by anyone at

any time. Once, again, please make function(s) that implements the following:  
most\_common\_byte(src\_addr, size): b = 0 i = 0 for i <= size-1: curr\_byte = [src\_addr + i] [stack\_base - curr\_byte] += 1 b = 0

```
max_freq = 0
max_freq_byte = 0
for b <= 0xff:
    if [stack_base - b] > max_freq:
        max_freq = [stack_base - b]
        max_freq_byte = b

return max_freq_byte
```

Assumptions: - There will never be more than 0xffff of any byte - The size will never be longer than 0xffff - The list will have at least one element Constraints: - You must put the “counting list” on the stack - You must restore the stack like in a normal function - You cannot modify the data at src\_addr

Please give me your assembly in bytes (up to 0x1000 bytes):

## INJECTION (Babyshell)

### 1

Welcome to /challenge/babyshell\_level1!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Placing shellcode on the stack at 0x7ffdf4927190! In this challenge, shellcode will be copied onto the stack and executed. Since the stack location is randomized on every execution, your shellcode will need to be *position-independent*.

Reading 0x1000 bytes from stdin.

### 2

Welcome to /challenge/babyshell\_level2!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure

that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors  $> 2$ .

[LEAK] Placing shellcode on the stack at 0x7ffd4bd23f60! In this challenge, shellcode will be copied onto the stack and executed. Since the stack location is randomized on every execution, your shellcode will need to be *position-independent*.

Reading 0x1000 bytes from stdin.

### 3

Welcome to /challenge/babysHELL\_level3!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors  $> 2$ .

[LEAK] Mapping shellcode memory at 0x1ea9d000! Reading 0x1000 bytes from stdin.

### 4

Welcome to /challenge/babysHELL\_level3!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors  $> 2$ .

[LEAK] Mapping shellcode memory at 0x1ea9d000! Reading 0x1000 bytes from stdin.

### 5

Welcome to /challenge/babysHELL\_level5!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors  $> 2$ .

[LEAK] Mapping shellcode memory at 0x1f1e2000!

## 6

Welcome to /challenge/babysHELL\_level5!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x1f1e2000!

## 7

Welcome to /challenge/babysHELL\_level7!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x21485000!

## 8

Welcome to /challenge/babysHELL\_level8!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x30326000! Reading 0x12 bytes from stdin.

## 9

Welcome to /challenge/babysHELL\_level9!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x30189000! Reading 0x1000 bytes from stdin.

## 10

Welcome to /challenge/babysHELL\_level10!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x22041000! Reading 0x1000 bytes from stdin.

## 11

Welcome to /challenge/babysHELL\_level11!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x25f42000! Reading 0x1000 bytes from stdin.

## 12

Welcome to /challenge/babysHELL\_level12!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x1d9e1000! Reading 0x1000 bytes from stdin.

## 13

Welcome to /challenge/babysHELL\_level13!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors  $> 2$ .

[LEAK] Mapping shellcode memory at 0x2dd5e000! Reading 0xc bytes from stdin.

## 14

Welcome to /challenge/babysHELL\_level14!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors  $> 2$ .

[LEAK] Mapping shellcode memory at 0x1d8cf000! Reading 0x6 bytes from stdin.

# MEMORY ERRORS (Babymem)

## 1.0

Welcome to /challenge/babymem\_level1.0!

The challenge() function has just been launched! Before we do anything, let's take a look at challenge()'s stack frame: +-----+

| Stack location                  | Data (bytes)            | Data (LE int)      |
|---------------------------------|-------------------------|--------------------|
| 0x00007ffd878b36b0 (rsp+0x0000) | 40 e5 00 a3 2b 7f 00 00 | 0x00007f2ba300e540 |
| 0x00007ffd878b36b8 (rsp+0x0008) | 58 48 8b 87 fd 7f 00 00 | 0x00007f2ba300e548 |
| 0x00007ffd878b36c0 (rsp+0x0010) | 48 48 8b 87 fd 7f 00 00 | 0x00007f2ba300e550 |
| 0x00007ffd878b36c8 (rsp+0x0018) | 00 00 00 00 01 00 00    | 0x00007f2ba300e558 |
| 0x00007ffd878b36d0 (rsp+0x0020) | a0 44 00 a3 2b 7f 00 00 | 0x00007f2ba300e560 |
| 0x00007ffd878b36d8 (rsp+0x0028) | 00 00 00 00 00 00 00 00 | 0x00007f2ba300e568 |
| 0x00007ffd878b36e0 (rsp+0x0030) | f0 36 8b 87 fd 7f 00 00 | 0x00007f2ba300e570 |
| 0x00007ffd878b36e8 (rsp+0x0038) | 04 37 8b 87 fd 7f 00 00 | 0x00007f2ba300e578 |
| 0x00007ffd878b36f0 (rsp+0x0040) | 00 00 00 00 00 00 00 00 | 0x00007f2ba300e580 |
| 0x00007ffd878b36f8 (rsp+0x0048) | 00 00 00 00 00 00 00 00 | 0x00007f2ba300e588 |
| 0x00007ffd878b3700 (rsp+0x0050) | 00 00 00 00 00 00 00 00 | 0x00007f2ba300e590 |
| 0x00007ffd878b3708 (rsp+0x0058) | 00 75 3b 12 16 c2 15 26 | 0x2615c216123b7500 |

```

0x00007ffd878b3710 (rsp+0x0060) | 50 47 8b 87 fd 7f 00 00 | 0x00007ffd878b4750 |
| 0x00007ffd878b3718 (rsp+0x0068) | 1f 88 3e 53 b5 55 00 00 | 0x000055b5533e881f
| +-----+-----+-----+ Our stack

```

pointer points to 0x7ffd878b36b0, and our base pointer points to 0x7ffd878b3710. This means that we have (decimal) 14 8-byte words in our stack frame, including the saved base pointer and the saved return address, for a total of 112 bytes. The input buffer begins at 0x7ffd878b36f0, partway through the stack frame, (“above” it in the stack are other local variables used by the function). Your input will be read into this buffer. The buffer is 20 bytes long, but the program will let you provide an arbitrarily large input length, and thus overflow the buffer.

In this level, there is a “win” variable. By default, the value of this variable is zero. However, when this variable is non-zero, the flag will be printed. You can make this variable be non-zero by overflowing the input buffer. The “win” variable is stored at 0x7ffd878b3704, 20 bytes after the start of your input buffer.

Payload size:

## 1.1

Welcome to /challenge/babymem\_level1.1!

Payload size:

## 2.0

Welcome to /challenge/babymem\_level2.0!

The challenge() function has just been launched! This challenge stores your input buffer on the heap! It also stores the “win” variable on the heap. Allocating memory for the input buffer... Called malloc(47) = 0x555fa92216b0 Called malloc(0x10) = 0x555fa92216f0 Called malloc(0x10) = 0x555fa9221710 Called malloc(0x10) = 0x555fa9221730 Called malloc(0x10) = 0x555fa9221750 Called malloc(0x10) = 0x555fa9221770 Called malloc(0x10) = 0x555fa9221790 Allocating memory for the win variable... Called calloc(1, sizeof(int)) = 0x555fa92217b0 In this level, there is a “win” variable. By default, the value of this variable is zero. However, when this variable is non-zero, the flag will be printed. You can make this variable be non-zero by overflowing the input buffer. The “win” variable is stored at 0x555fa92217b0, 256 bytes after the start of your input buffer.

Payload size:

## 2.1

Welcome to /challenge/babymem\_level2.1!

Payload size:

### 3.0

Welcome to /challenge/babymem\_level3.0!

The challenge() function has just been launched! Before we do anything, let's take a look at challenge()'s stack frame: +-----+

| +-----+                                                                   | Stack location                  | Data (bytes)            | Data (LE int)                   |
|---------------------------------------------------------------------------|---------------------------------|-------------------------|---------------------------------|
| +-----+                                                                   | +-----+                         | +-----+                 | +-----+                         |
| 0x00007ffe89b1fd50 (rsp+0x0000)                                           | 00 00 00 00 00 00 00 00         | 0x0000000000000000      |                                 |
| 0x00007ffe89b1fd58 (rsp+0x0008)                                           | 48 0f b2 89 fe 7f 00 00         | 0x00007ffe89b20f48      |                                 |
| 0x00007ffe89b1fd60 (rsp+0x0010)                                           | 38 0f b2 89 fe 7f 00 00         | 0x00007ffe89b20f38      |                                 |
| 0x00007ffe89b1fd68 (rsp+0x0018)                                           | 00 ca 5b f6 01 00 00 00         |                         |                                 |
| 0x00000001f65bca00                                                        | 0x00007ffe89b1fd70 (rsp+0x0020) | 00 00 00 00 00 00 00 00 |                                 |
| 00   0x0000000000000000                                                   | 0x00007ffe89b1fd78 (rsp+0x0028) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00   0x0000000000000000                                                | 0x00007ffe89b1fd80 (rsp+0x0030) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00   0x0000000000000000                                             | 0x00007ffe89b1fd88 (rsp+0x0038) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00   0x0000000000000000                                          | 0x00007ffe89b1fd90 (rsp+0x0040) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00   0x0000000000000000                                       | 0x00007ffe89b1fd98 (rsp+0x0048) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00 00   0x0000000000000000                                    | 0x00007ffe89b1fda0 (rsp+0x0050) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00 00 00   0x0000000000000000                                 | 0x00007ffe89b1fda8 (rsp+0x0058) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00 00 00 00   0x0000000000000000                              | 0x00007ffe89b1fdb0 (rsp+0x0060) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00 00 00 00 00   0x0000000000000000                           | 0x00007ffe89b1fdb8 (rsp+0x0068) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00 00 00 00 00 00   0x0000000000000000                        | 0x00007ffe89b1fdc0 (rsp+0x0070) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00 00 00 00 00 00 00   0x0000000000000000                     | 0x00007ffe89b1fdc8 (rsp+0x0078) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00 00 00 00 00 00 00 00   0x0000000000000000                  | 0x00007ffe89b1fdd0 (rsp+0x0080) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00 00 00 00 00 00 00 00 00   0x0000000000000000               | 0x00007ffe89b1fdd8 (rsp+0x0088) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00 00 00 00 00 00 00 00 00 00   0x0000000000000000            | 0x00007ffe89b1fde0 (rsp+0x0090) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   0x0000000000000000         | 0x00007ffe89b1fde8 (rsp+0x0098) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   0x0000000000000000      | 0x00007ffe89b1fdf0 (rsp+0x00a0) | 00 00 00 00 00 00 00 00 |                                 |
| 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   0x0000000000000000   | 0x00007ffe89b1fdf8 (rsp+0x00a8) | 40 0e b2 89 fe 7f 00 00 | 0x00007ffe89b1fe00 (rsp+0x00b0) |
| d0 11 40 00 00 00 00 00 00 00 00 00 00 00 00 00 00   0x0000000000004011d0 | 0x00007ffe89b1fe08 (rsp+0x00b8) | 80 fd b1 89 fe 7f 00 00 | 0x00007ffe89b1fd80              |
| 0x00007ffe89b1fe10 (rsp+0x00c0)                                           | 40 0e b2 89 fe 7f 00 00         | 0x00007ffe89b20e40      | 0x00007ffe89b1fe18 (rsp+0x00c8) |
| b0 26 40 00 00 00 00 00 00 00 00 00 00 00 00 00 00                        | 0x0000000000004026b0            |                         |                                 |

+-----+ Our stack pointer points to 0x7ffe89b1fd50, and our base pointer points to 0x7ffe89b1fe10. This means that we have (decimal) 26 8-byte words in our stack frame, including the saved base pointer and the saved return address, for a total of 208 bytes. The input buffer begins at 0x7ffe89b1fd80, partway through the stack frame, ("above" it in the stack are other local variables used by the function). Your input will be read into this buffer. The buffer is 119 bytes long, but the program will let you provide an arbitrarily large input length, and thus overflow the buffer.

In this level, there is no "win" variable. You will need to force the program to

execute the `win()` function by directly overflowing into the stored return address back to `main`, which is stored at `0x7ffe89b1fe18`, 152 bytes after the start of your input buffer. That means that you will need to input at least 160 bytes (119 to fill the buffer, 33 to fill other stuff stored between the buffer and the return address, and 8 that will overwrite the return address).

We have disabled the following standard memory corruption mitigations for this challenge: - the canary is disabled, otherwise you would corrupt it before overwriting the return address, and the program would abort. - the binary is *not* position independent. This means that it will be located at the same spot every time it is run, which means that by analyzing the binary (using `objdump` or reading this output), you can know the exact value that you need to overwrite the return address with.

Payload size:

### 3.1

Welcome to `/challenge/babymem_level3.1!`

Payload size:

### 4.0

Welcome to `/challenge/babymem_level4.0!`

The `challenge()` function has just been launched! Before we do anything, let's take a look at `challenge()`'s stack frame: +-----+

| +-----+                        | Stack location                  | Data (bytes)            | Data (LE int)                                                                                                                                                             |
|--------------------------------|---------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| +-----+                        |                                 |                         |                                                                                                                                                                           |
| 0x00007ffc21adfb0 (rsp+0x0000) | 40 95 4a c1 cb 7f 00 00         | 0x00007fcb14a9540       |                                                                                                                                                                           |
| 0x00007ffc21adfb8 (rsp+0x0008) | 38 0d ae 21 fc 7f 00 00         | 0x00007ffc21ae0d38      |                                                                                                                                                                           |
| 0x00007ffc21adfb0 (rsp+0x0010) | 28 0d ae 21 fc 7f 00 00         | 0x00007ffc21ae0d28      |                                                                                                                                                                           |
| 0x00007ffc21adfb8 (rsp+0x0018) | 00 00 00 00 01 00 00 00         |                         |                                                                                                                                                                           |
| 0x0000000100000000             | 0x00007ffc21adfb0 (rsp+0x0020)  | a0 f4 49 c1 cb 7f 00 00 |                                                                                                                                                                           |
| 0x00007fcb149f4a0              | 0x00007ffc21adfb8 (rsp+0x0028)  | 3d 45 34 c1 00 00 00 00 |                                                                                                                                                                           |
| 0x00000000c134453d             | 0x00007ffc21adfb0 (rsp+0x0030)  | 00 00 00 00 00 00 00 00 |                                                                                                                                                                           |
| 0x0000000000000000             | 0x00007ffc21adfb8 (rsp+0x0038)  | 00 00 00 00 00 00 00 00 |                                                                                                                                                                           |
| 0x0000000000000000             | 0x00007ffc21adfbe0 (rsp+0x0040) | 00 00 00 00 00 00 00 00 |                                                                                                                                                                           |
| 0x0000000000000000             | 0x00007ffc21adfbe8 (rsp+0x0048) | 00 00 00 21 fc 7f 00 00 |                                                                                                                                                                           |
| 0x00007ffc21000000             | 0x00007ffc21adfb0 (rsp+0x0050)  | d0 11 40 00 00 00 00 00 |                                                                                                                                                                           |
| 0x0000000004011d0              | 0x00007ffc21adfb8 (rsp+0x0058)  | d0 fb ad 21 fc 7f 00 00 |                                                                                                                                                                           |
| 0x00007ffc21adfb0              | 0x00007ffc21adfb0 (rsp+0x0060)  | 30 0c ae 21 fc 7f 00 00 |                                                                                                                                                                           |
| 0x00007ffc21ae0c30             | 0x00007ffc21adfb8 (rsp+0x0068)  | 63 1d 40 00 00 00 00 00 |                                                                                                                                                                           |
| 0x000000000401d63              |                                 |                         |                                                                                                                                                                           |
| +-----+                        |                                 |                         |                                                                                                                                                                           |
|                                |                                 |                         | Our stack pointer points to 0x7ffc21adfb0, and our base pointer points to 0x7ffc21adfc00. This means that we have (decimal) 14 8-byte words in our stack frame, including |

the saved base pointer and the saved return address, for a total of 112 bytes. The input buffer begins at 0x7ffc21adfbdb0, partway through the stack frame, (“above” it in the stack are other local variables used by the function). Your input will be read into this buffer. The buffer is 27 bytes long, but the program will let you provide an arbitrarily large input length, and thus overflow the buffer.

In this level, there is no “win” variable. You will need to force the program to execute the win() function by directly overflowing into the stored return address back to main, which is stored at 0x7ffc21adfc08, 56 bytes after the start of your input buffer. That means that you will need to input at least 64 bytes (27 to fill the buffer, 29 to fill other stuff stored between the buffer and the return address, and 8 that will overwrite the return address).

We have disabled the following standard memory corruption mitigations for this challenge: - the canary is disabled, otherwise you would corrupt it before overwriting the return address, and the program would abort. - the binary is *not* position independent. This means that it will be located at the same spot every time it is run, which means that by analyzing the binary (using objdump or reading this output), you can know the exact value that you need to overwrite the return address with.

Payload size:

## 4.1

Welcome to /challenge/babymem\_level4.1!

Payload size:

## 5.0

Welcome to /challenge/babymem\_level5.0!

The challenge() function has just been launched! Before we do anything, let’s take a look at challenge()’s stack frame: +-----+-----

| +-----+                         | Stack location                  | Data (bytes)                    | Data (LE int) |
|---------------------------------|---------------------------------|---------------------------------|---------------|
| +-----+                         |                                 |                                 |               |
| 0x00007ffe045f8230 (rsp+0x0000) | a0 04 d6 f7 3f 7f 00 00         | 0x00007f3ff7d604a0              |               |
| 0x00007ffe045f8238 (rsp+0x0008) | e8 93 5f 04 fe 7f 00 00         | 0x00007ffe045f93e8              |               |
| 0x00007ffe045f8240 (rsp+0x0010) | d8 93 5f 04 fe 7f 00 00         | 0x00007ffe045f93d8              |               |
| 0x00007ffe045f8248 (rsp+0x0018) | dd 95 c0 f7 01 00 00 00         | 0x00000001f7c095dd              |               |
| 0x00007ffe045f8250 (rsp+0x0020) | 40 a5 d6 f7 3f 7f 00 00         | 0x00007f3ff7d6a540              |               |
| 0x00007ffe045f8258 (rsp+0x0028) | a0 46 d6 f7 3f 7f 00 00         | 0x00007f3ff7d646a0              |               |
| 0x00007ffe045f8260 (rsp+0x0030) | 00 00 00 00 00 00 00 00         |                                 |               |
| 0x0000000000000000              | 0x00007ffe045f8268 (rsp+0x0038) | 00 00 00 00                     |               |
| 00 00 00 00                     | 0x0000000000000000              | 0x00007ffe045f8270 (rsp+0x0040) | 00 00 00      |
| 00 00 00 00                     | 0x0000000000000000              | 0x00007ffe045f8278 (rsp+0x0048) | 00 00         |

```

00 00 00 00 00 00 | 0x0000000000000000 | | 0x00007ffe045f8280 (rsp+0x0050) | 00
00 00 00 00 00 00 00 | 0x0000000000000000 | | 0x00007ffe045f8288 (rsp+0x0058)
| 00 00 00 00 00 00 00 00 | 0x0000000000000000 | | 0x00007ffe045f8290
(rsp+0x0060) | 80 2c 40 00 00 00 00 00 | 0x000000000000402c80 | |
0x00007ffe045f8298 (rsp+0x0068) | e0 92 5f 04 fe 7f 00 00 | 0x00007ffe045f92e0 | |
0x00007ffe045f82a0 (rsp+0x0070) | 00 00 00 00 00 00 00 00 | 0x0000000000000000
| | 0x00007ffe045f82a8 (rsp+0x0078) | 60 82 5f 04 fe 7f 00 00 | 0x00007ffe045f8260 |
| 0x00007ffe045f82b0 (rsp+0x0080) | e0 92 5f 04 fe 7f 00 00 | 0x00007ffe045f92e0 | |
0x00007ffe045f82b8 (rsp+0x0088) | 75 2c 40 00 00 00 00 00 | 0x000000000000402c75
| +-----+-----+-----+ Our stack
pointer points to 0x7ffe045f8230, and our base pointer points to 0x7ffe045f82b0.
This means that we have (decimal) 18 8-byte words in our stack frame, including
the saved base pointer and the saved return address, for a total of 144 bytes.
The input buffer begins at 0x7ffe045f8260, partway through the stack frame,
("above" it in the stack are other local variables used by the function). Your
input will be read into this buffer. The buffer is 47 bytes long, but the program
will let you provide an arbitrarily large input length, and thus overflow the
buffer.

```

In this level, there is no “win” variable. You will need to force the program to execute the win() function by directly overflowing into the stored return address back to main, which is stored at 0x7ffe045f82b8, 88 bytes after the start of your input buffer. That means that you will need to input at least 96 bytes (47 to fill the buffer, 41 to fill other stuff stored between the buffer and the return address, and 8 that will overwrite the return address).

We have disabled the following standard memory corruption mitigations for this challenge: - the canary is disabled, otherwise you would corrupt it before overwriting the return address, and the program would abort. - the binary is *not* position independent. This means that it will be located at the same spot every time it is run, which means that by analyzing the binary (using objdump or reading this output), you can know the exact value that you need to overwrite the return address with.

This challenge will let you send multiple payload records concatenated together. It will make sure that the total payload size fits in the allocated buffer on the stack. Can you send a carefully crafted input to break this calculation? Number of payload records to send:

## 5.1

Welcome to /challenge/babymem\_level5.1!

Number of payload records to send:

## 6.0

Welcome to /challenge/babymem\_level6.0!

The challenge() function has just been launched! Before we do anything, let's take a look at challenge()'s stack frame: +-----+

| +-----+   Stack location   Data (bytes)   Data (LE int) |                                                           |
|---------------------------------------------------------|-----------------------------------------------------------|
| +-----+   +-----+                                       |                                                           |
| 0x00007fff5dfc77c0 (rsp+0x0000)                         | 00 00 00 00 00 00 00 00   0x0000000000000000              |
| 0x00007fff5dfc77c8 (rsp+0x0008)                         | a8 89 fc 5d ff 7f 00 00   0x00007fff5dfc89a8              |
| 0x00007fff5dfc77d0 (rsp+0x0010)                         | 98 89 fc 5d ff 7f 00 00   0x00007fff5dfc8998              |
| 0x00007fff5dfc77d8 (rsp+0x0018)                         | a0 66 f9 d2 01 00 00 00   0x00000001d2f966a0              |
| 0x00007fff5dfc77e0 (rsp+0x0020)                         | 24 67 f9 d2 0c 7f 00 00   0x00007f0cd2f96724              |
| 0x00007fff5dfc77e8 (rsp+0x0028)                         | 00 00 00 00 00 00 00 00                                   |
| 0x0000000000000000                                      | 0x00007fff5dfc77f0 (rsp+0x0030)   00 00 00 00 00 00 00 00 |
| 0x0000000000000000                                      | 0x00007fff5dfc77f8 (rsp+0x0038)   00 00 00 00 00 00 00 00 |
| 00   0x0000000000000000                                 | 0x00007fff5dfc7800 (rsp+0x0040)   00 00 00 00 00 00 00 00 |
| 00 00   0x0000000000000000                              | 0x00007fff5dfc7808 (rsp+0x0048)   00 00 00 00 00 00 00 00 |
| 00 00 00   0x0000000000000000                           | 0x00007fff5dfc7810 (rsp+0x0050)   00 00 00 00 00 00 00 00 |
| 00 00 00 00   0x0000000000000000                        | 0x00007fff5dfc7818 (rsp+0x0058)   00 00 00 00 00 00 00 00 |
| 00 00 00 00 00   0x0000000000000000                     | 0x00007fff5dfc7820 (rsp+0x0060)   00 00 00 00 00 00 00 00 |
| 00 00 00 00 00 00   0x0000000000000000                  | 0x00007fff5dfc7828 (rsp+0x0068)   00 00 00 00 00 00 00 00 |
| 00 00 00 00 00 00 00   0x0000000000000000               | 0x00007fff5dfc7830 (rsp+0x0070)   00 00 00 00 00 00 00 00 |
| 00 00 00 00 00 00 00 00   0x0000000000000000            | 0x00007fff5dfc7838 (rsp+0x0078)   00 00 00 00 00 00 00 00 |
| 0x00007fff5dfc7840 (rsp+0x0080)                         | 00 00 00 00 00 00 00 00   0x0000000000000000              |
| 0x00007fff5dfc7848 (rsp+0x0088)                         | 00 00 00 00 00 00 00 00   0x0000000000000000              |
| 0x00007fff5dfc7850 (rsp+0x0090)                         | 00 00 00 00 00 00 00 00   0x0000000000000000              |
| 0x00007fff5dfc7858 (rsp+0x0098)                         | 00 00 00 5d ff 7f 00 00   0x00007fff5d000000              |
| 0x00007fff5dfc7860 (rsp+0x00a0)                         | d0 11 40 00 00 00 00 00   0x00000000004011d0              |
| 0x00007fff5dfc7868 (rsp+0x00a8)                         | f0 77 fc 5d ff 7f 00 00   0x00007fff5dfc77f0              |
| 0x00007fff5dfc7870 (rsp+0x00b0)                         | a0 88 fc 5d ff 7f 00 00   0x00007fff5dfc88a0              |
| 0x00007fff5dfc7878 (rsp+0x00b8)                         | f2 2a 40 00 00 00 00 00   0x0000000000402af2              |
| +-----+   +-----+   Our stack                           |                                                           |

pointer points to 0x7fff5dfc77c0, and our base pointer points to 0x7fff5dfc7870. This means that we have (decimal) 24 8-byte words in our stack frame, including the saved base pointer and the saved return address, for a total of 192 bytes. The input buffer begins at 0x7fff5dfc77f0, partway through the stack frame, (“above” it in the stack are other local variables used by the function). Your input will be read into this buffer. The buffer is 107 bytes long, but the program will let you provide an arbitrarily large input length, and thus overflow the buffer.

In this level, there is no “win” variable. You will need to force the program to execute the `win_authed()` function by directly overflowing into the stored return address back to main, which is stored at `0x7fff5dfc7878`, 136 bytes after the start of your input buffer. That means that you will need to input at least 144 bytes (107 to fill the buffer, 29 to fill other stuff stored between the buffer and the return address, and 8 that will overwrite the return address).

We have disabled the following standard memory corruption mitigations for

this challenge: - the canary is disabled, otherwise you would corrupt it before overwriting the return address, and the program would abort. - the binary is *not* position independent. This means that it will be located at the same spot every time it is run, which means that by analyzing the binary (using objdump or reading this output), you can know the exact value that you need to overwrite the return address with.

Payload size:

## 6.1

Welcome to /challenge/babymem\_level6.1!

Payload size:

## 7.0

Welcome to /challenge/babymem\_level7.0!

The challenge() function has just been launched! Before we do anything, let's take a look at challenge()'s stack frame: +-----+

| +-----+                         | Stack location                  | Data (bytes)            | Data (LE int) |
|---------------------------------|---------------------------------|-------------------------|---------------|
| +-----+                         |                                 |                         |               |
| 0x00007ffde49bc200 (rsp+0x0000) | ff ff ff 00 00 00 00            | 0x00000000ffffff        |               |
| 0x00007ffde49bc208 (rsp+0x0008) | e8 d3 9b e4 fd 7f 00 00         | 0x00007ffde49bd3e8      |               |
| 0x00007ffde49bc210 (rsp+0x0010) | d8 d3 9b e4 fd 7f 00 00         | 0x00007ffde49bd3d8      |               |
| 0x00007ffde49bc218 (rsp+0x0018) | a0 86 7d f6 01 00 00 00         |                         |               |
| 0x00000001f67d86a0              | 0x00007ffde49bc220 (rsp+0x0020) | 24 87 7d f6 fc 7e 00 00 |               |
| 0x00007efcf67d8724              | 0x00007ffde49bc228 (rsp+0x0028) | 00 00 00 00 00 00 00 00 |               |
| 0x0000000000000000              | 0x00007ffde49bc230 (rsp+0x0030) | 00 00 00 00 00 00 00 00 |               |
| 0x0000000000000000              | 0x00007ffde49bc238 (rsp+0x0038) | 00 00 00 00 00 00 00 00 |               |
| 0x0000000000000000              | 0x00007ffde49bc240 (rsp+0x0040) | 00 00 00 00 00 00 00 00 |               |
| 0x0000000000000000              | 0x00007ffde49bc248 (rsp+0x0048) | 00 00 00 00 00 00 00 00 |               |
| 0x0000000000000000              | 0x00007ffde49bc250 (rsp+0x0050) | 00 00 00 00 00 00 00 00 |               |
| 0x0000000000000000              | 0x00007ffde49bc258 (rsp+0x0058) | 00 00 00 00 00 00 00 00 |               |
| 0x0000000000000000              | 0x00007ffde49bc260 (rsp+0x0060) | 00 00 00 00 00 00 00 00 |               |
| 0x00007ffde49bc268 (rsp+0x0068) | 00 00 00 00 00 00 00 00         | 0x0000000000000000      |               |
| 0x00007ffde49bc270 (rsp+0x0070) | 00 00 00 00 00 00 00 00         |                         |               |
| 0x0000000000000000              | 0x00007ffde49bc278 (rsp+0x0078) | 00 00 00 00 00 00 00 00 |               |
| 0x0000000000000000              | 0x00007ffde49bc280 (rsp+0x0080) | 00 00 00 00 00 00 00 00 |               |
| 0x0000000000000000              | 0x00007ffde49bc288 (rsp+0x0088) | 00 00 00 00 00 00 00 00 |               |
| 0x0000000000000000              | 0x00007ffde49bc290 (rsp+0x0090) | 00 00 00 00 00 00 00 00 |               |
| 0x0000000000000000              | 0x00007ffde49bc298 (rsp+0x0098) | 00 00 00 00 00 00 00 00 |               |
| 0x0000000000000000              | 0x00007ffde49bc2a0 (rsp+0x00a0) | e0 f1 96 9b b2 55 00 00 |               |
| 0x000055b29b96f1e0              | 0x00007ffde49bc2a8 (rsp+0x00a8) | 30 c2 9b e4 fd 7f 00 00 |               |
| 0x00007ffde49bc230              | 0x00007ffde49bc2b0 (rsp+0x00b0) |                         |               |

```

| e0 d2 9b e4 fd 7f 00 00 | 0x00007ffde49bd2e0 | | 0x00007ffde49bc2b8
(rsp+0x00b8) | 48 0b 97 9b b2 55 00 00 | 0x000055b29b970b48 |
+-----+-----+-----+
Our stack
pointer points to 0x7ffde49bc200, and our base pointer points to 0x7ffde49bc2b0.
This means that we have (decimal) 24 8-byte words in our stack frame, including
the saved base pointer and the saved return address, for a total of 192 bytes.
The input buffer begins at 0x7ffde49bc230, partway through the stack frame,
("above" it in the stack are other local variables used by the function). Your
input will be read into this buffer. The buffer is 112 bytes long, but the program
will let you provide an arbitrarily large input length, and thus overflow the
buffer.

```

In this level, there is no “win” variable. You will need to force the program to execute the `win_authed()` function by directly overflowing into the stored return address back to main, which is stored at `0x7ffde49bc2b8`, 136 bytes after the start of your input buffer. That means that you will need to input at least 144 bytes (112 to fill the buffer, 24 to fill other stuff stored between the buffer and the return address, and 8 that will overwrite the return address).

We have disabled the following standard memory corruption mitigations for this challenge: - the canary is disabled, otherwise you would corrupt it before overwriting the return address, and the program would abort. Because the binary is position independent, you cannot know exactly where the `win_authed()` function is located. This means that it is not clear what should be written into the return address.

Payload size:

## 7.1

Welcome to /challenge/babymem\_level7.1!

Payload size:

## 8.0

Welcome to /challenge/babymem\_level8.0!

The `challenge()` function has just been launched! Before we do anything, let’s take a look at `challenge()`’s stack frame:

| Stack location                  | Data (bytes)            | Data (LE int)       |
|---------------------------------|-------------------------|---------------------|
| 0x00007ffcf18d3800 (rsp+0x0000) | 10 38 8d f1 fc 7f 00 00 | 0x00007ffcf18d3810  |
| 0x00007ffcf18d3808 (rsp+0x0008) | f8 49 8d f1 fc 7f 00 00 | 0x00007ffcf18d49f8  |
| 0x00007ffcf18d3810 (rsp+0x0010) | e8 49 8d f1 fc 7f 00 00 | 0x00007ffcf18d49e8  |
| 0x00007ffcf18d3818 (rsp+0x0018) | 00 1c 2c 1c 01 00 00 00 | 0x0000000011c2c1c00 |
| 0x00007ffcf18d3820 (rsp+0x0020) | 01 00 00 00 00 00 00 00 | 0x0000000000000001  |
| 0x00007ffcf18d3828 (rsp+0x0028) | 00 00 00 00             |                     |

```

00 00 00 00 | 0x0000000000000000 || 0x00007ffcf18d3830 (rsp+0x0030) | 00 00 00
00 00 00 00 00 | 0x0000000000000000 || 0x00007ffcf18d3838 (rsp+0x0038) | 00 00
00 00 00 00 00 00 | 0x0000000000000000 || 0x00007ffcf18d3840 (rsp+0x0040) | 00
00 00 00 00 00 00 00 | 0x0000000000000000 || 0x00007ffcf18d3848 (rsp+0x0048)
| 00 00 00 00 00 00 00 00 | 0x0000000000000000 || 0x00007ffcf18d3850
(rsp+0x0050) | 00 00 00 00 00 00 00 00 | 0x0000000000000000 ||
0x00007ffcf18d3858 (rsp+0x0058) | 00 00 00 00 00 00 00 00 | 0x0000000000000000
| | 0x00007ffcf18d3860 (rsp+0x0060) | 00 00 00 00 00 00 00 00 |
0x0000000000000000 || 0x00007ffcf18d3868 (rsp+0x0068) | 00 00 00 00 00 00 00
00 | 0x0000000000000000 || 0x00007ffcf18d3870 (rsp+0x0070) | 00 00 00 00 00 00
00 00 | 0x0000000000000000 || 0x00007ffcf18d3878 (rsp+0x0078) | 00 00 00 00 00
00 00 00 | 0x0000000000000000 || 0x00007ffcf18d3880 (rsp+0x0080) | 00 00 00 00
00 00 00 00 | 0x0000000000000000 || 0x00007ffcf18d3888 (rsp+0x0088) | 00 00 00
00 00 00 00 00 | 0x0000000000000000 || 0x00007ffcf18d3890 (rsp+0x0090) | 00
00 00 00 00 00 00 00 | 0x0000000000000000 || 0x00007ffcf18d3898 (rsp+0x0098)
| e5 ed 18 79 37 7f 00 00 | 0x00007f377918ede5 | | 0x00007ffcf18d38a0
(rsp+0x00a0) | 00 a6 ba e1 60 55 00 00 | 0x00005560e1baa600 | |
0x00007ffcf18d38a8 (rsp+0x00a8) | f0 48 8d f1 fc 7f 00 00 | 0x00007ffcf18d48f0 | |
0x00007ffcf18d38b0 (rsp+0x00b0) | 40 92 ba e1 60 55 00 00 | 0x00005560e1ba9240
| | 0x00007ffcf18d38b8 (rsp+0x00b8) | 30 38 8d f1 fc 7f 00 00 | 0x00007ffcf18d3830
| | 0x00007ffcf18d38c0 (rsp+0x00c0) | f0 48 8d f1 fc 7f 00 00 | 0x00007ffcf18d48f0 |
| 0x00007ffcf18d38c8 (rsp+0x00c8) | f4 a5 ba e1 60 55 00 00 | 0x00005560e1baa5f4
| +-----+ -+-----+

```

Our stack pointer points to 0x7ffcf18d3800, and our base pointer points to 0x7ffcf18d38c0. This means that we have (decimal) 26 8-byte words in our stack frame, including the saved base pointer and the saved return address, for a total of 208 bytes. The input buffer begins at 0x7ffcf18d3830, partway through the stack frame, (“above” it in the stack are other local variables used by the function). Your input will be read into this buffer. The buffer is 103 bytes long, but the program will let you provide an arbitrarily large input length, and thus overflow the buffer.

In this level, there is no “win” variable. You will need to force the program to execute the win\_authed() function by directly overflowing into the stored return address back to main, which is stored at 0x7ffcf18d38c8, 152 bytes after the start of your input buffer. That means that you will need to input at least 160 bytes (103 to fill the buffer, 49 to fill other stuff stored between the buffer and the return address, and 8 that will overwrite the return address).

We have disabled the following standard memory corruption mitigations for this challenge: - the canary is disabled, otherwise you would corrupt it before overwriting the return address, and the program would abort. Because the binary is position independent, you cannot know exactly where the win\_authed() function is located. This means that it is not clear what should be written into the return address.

Payload size:

## 8.1

Welcome to /challenge/babymem\_level8.1!

Payload size: